BUGPILOT

Secrets Management & Client-Side Exposure
Security Review and Remediation Report

Report detail	Description
Project	BugPilot – AI-Powered Bug Tracking System
Technology stack	React.js + Vite, FastAPI, PostgreSQL 16
Review focus	API keys, private tokens, environment variables, client bundle exposure
Assessment date	30 October 2026
Assessment status	Guidance and preliminary review based on shared project context; source scan not completed

Purpose: Explain how to detect accidental exposure of secrets in a React/Vite application and move private credentials to the FastAPI backend or a suitable secrets-management system.

Handling note: Do not place real credentials in this report or upload them for review. Use placeholders when sharing configuration.

1. Executive Summary

This report addresses the risk of exposing API keys, private tokens, database credentials, signing secrets, and sensitive environment variables through BugPilot's React frontend or its built JavaScript assets.

The key rule is that frontend code runs on a user's device. Anything embedded in a browser-delivered JavaScript bundle, HTML file, source map, or network response should be treated as visible to that user. Environment-variable prefixes do not make a value secret; they determine whether a tool exposes it to client code.

Assessment limitation: No complete project archive, package files, environment files, built assets, or Git history were supplied for this specific scan. Therefore, this report does not claim that a particular secret was found or that the repository has been fully scanned. It provides a concrete scan procedure and secure migration examples.

2. BugPilot Architecture and Trust Boundary

BugPilot uses React + Vite for the browser frontend, FastAPI for API operations, and PostgreSQL for persistent storage. The browser is an untrusted environment: users can inspect downloaded JavaScript, HTML, network requests, browser storage, and client configuration.

Recommended flow: React browser → HTTPS → FastAPI → PostgreSQL or approved external service.

Private credentials should be held and used by FastAPI or a managed secrets service. React should call a BugPilot backend endpoint rather than call a private third-party API with a secret embedded in the frontend.

3. What Counts as a Secret?

Value	Secret?	Correct location
PostgreSQL username/password or DATABASE_URL	Yes	Backend environment / secret manager
JWT signing key or session encryption key	Yes	Backend environment / secret manager
Private third-party API key	Yes	Backend environment / secret manager
OAuth client secret	Yes	Backend environment / secret manager
User access token / refresh token	Sensitive credential	Secure session design; never hardcode in frontend
Public API base URL	Usually no	Vite public configuration is acceptable
Public analytics/project identifier	Depends on provider	May be public, but apply provider restrictions

4. Vite Environment Variable Prefixes

Vite exposes variables with the `VITE_` prefix to code running in the browser. These values are included in the client-side build and must not contain secrets.

4.1 Safe example – public configuration

```
# frontend/.env
VITE_API_BASE_URL=http://localhost:8000/api

// src/services/bugService.js
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

A backend URL is normally public configuration: users can see it in browser network tools regardless. In production, configure the HTTPS API URL.

4.2 Unsafe example – private key in Vite

```
# NEVER put a private key in a VITE_ variable
VITE_STRIPE_SECRET_KEY=sk_live_REPLACE_ME
VITE_DATABASE_PASSWORD=REPLACE_ME
VITE_JWT_SECRET=REPLACE_ME
```

These values are exposed to the frontend build because of the `VITE_` prefix. Renaming a variable to another frontend-readable name does not make a secret safe if the frontend imports and bundles it.

4.3 Non-prefixed frontend variables

A non-prefixed variable in a Vite `.env` file is not automatically exposed through `import.meta.env` by default. However, do not treat that as a secret-management solution if the value is later imported, copied into source code, injected by custom build configuration, or otherwise sent to the browser. Keep private variables in the backend environment, not the frontend project.

5. Examples of Accidental Exposure

5.1 Hardcoded secret in React source

```
// UNSAFE – src/services/paymentService.js
const API_KEY = "sk_live_abc123";
fetch("https://payments.example/api", {
  headers: { Authorization: `Bearer ${API_KEY}` }
});
```

Why unsafe: the key becomes part of the downloaded JavaScript and can be extracted from the browser or build output.

5.2 Database credentials in frontend configuration

```
// UNSAFE – any React source file
const DATABASE_URL =
  "postgresql://admin:password@db-host:5432/bugpilot";
```

Why unsafe: a browser application must never connect directly to PostgreSQL using embedded database credentials. All database access should go through FastAPI.

5.3 Secret included in a frontend request

```
// UNSAFE – private service key sent from browser
fetch("https://vendor.example/api/data", {
  headers: { "X-API-Key": import.meta.env.VITE_VENDOR_SECRET }
});
```

Why unsafe: users can inspect request headers in browser developer tools, even if the key was not obvious in the source.

6. Secure Pattern – Move Private API Calls to FastAPI

Instead of calling a third-party service directly from React with a private key, React should call a BugPilot backend route. FastAPI reads the secret from its server-side environment and makes the external request.

6.1 Backend environment file

```
# backend/.env (DO NOT COMMIT)
VENDOR_API_KEY=replace_with_real_secret
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@localhost:5432/bugpilot_db
```

6.2 Load backend environment variables

```
# backend/app/config.py
import os
from dotenv import load_dotenv

load_dotenv()

VENDOR_API_KEY = os.getenv("VENDOR_API_KEY")
DATABASE_URL = os.getenv("DATABASE_URL")

if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL is not configured")
```

Install python-dotenv in the backend virtual environment and record it in backend/requirements.txt:

```
python -m pip install python-dotenv
# Add this line to backend/requirements.txt:
python-dotenv
```

6.3 FastAPI endpoint uses the secret server-side

```
# backend/app/main.py – illustrative pattern
from fastapi import APIRouter, HTTPException
import httpx
from app.config import VENDOR_API_KEY

router = APIRouter()
```

```
@router.get("/api/vendor-data")
async def get_vendor_data():
    if not VENDOR_API_KEY:
        raise HTTPException(
            status_code=500,
            detail="Service is not configured"
        )

    async with httpx.AsyncClient(timeout=10.0) as client:
        response = await client.get(
            "https://vendor.example/api/data",
            headers={"X-API-Key": VENDOR_API_KEY}
        )
        response.raise_for_status()
        return response.json()
```

This is an architectural example, not a drop-in endpoint for a particular vendor. Add vendor-specific error handling, response validation, authorization, rate limiting, and timeout/retry policy as appropriate. Do not blindly proxy arbitrary user-supplied URLs.

6.4 React calls only the BugPilot backend

```
// frontend/src/services/vendorService.js
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

export async function getVendorData() {
  const response = await fetch(
    `${API_BASE_URL}/vendor-data`
  );

  if (!response.ok) {
    throw new Error("Unable to load vendor data");
  }

  return response.json();
}
```

The browser sees the BugPilot API URL and response, but it does not receive the vendor's private API key.

7. Protect Environment Files and Git

7.1 Suggested project layout

```
ai-bug-tracker/
■■■ src/
■■■ backend/
■   ■■■ app/
■   ■■■ .env             # private backend values
■   ■■■ .env.example    # placeholders only
■   ■■■ requirements.txt
■■■ .gitignore
■■■ package.json
```

7.2 Root .gitignore

```
# Environment and secret files
.env
.env.*
!.env.example
backend/.env
frontend/.env

# Dependencies and generated files
node_modules/
backend/.venv/
dist/
__pycache__/
*.pyc
```

Adapt these paths to the actual repository layout. If a frontend `.env` contains only public VITE_ settings, it may be committed in some workflows, but ignoring local environment files by default reduces accidental commits. Keep a sanitized example file.

7.3 Example configuration

```
# backend/.env.example – safe placeholders
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@localhost:5432/bugpilot_db
VENDOR_API_KEY=replace_me
```

Never put actual credentials in `.env.example`.

8. Scan Procedure – Source, Configuration, Build and Git

Run these checks from the project root in PowerShell. Review each match manually; search results can include harmless variable names or documentation.

8.1 Search source and configuration for likely secret names

```
Get-ChildItem . -Recurse -File -Exclude package-lock.json |
  Where-Object {
    $_.FullName -notmatch '\\node_modules\\|\\.venv\\|\\dist\\'
  } |
  Select-String -Pattern
  'API_KEY|SECRET|TOKEN|PASSWORD|DATABASE_URL|PRIVATE_KEY|CLIENT_SECRET'
```

Do not print or paste actual secret values into chat. If a match is suspicious, redact the value and share only the file path and variable name.

8.2 Search Vite-prefixed variables

```
Get-ChildItem . -Recurse -File -Include *.env,*.env.* |
  Where-Object { $_.FullName -notmatch '\\node_modules\\|\\.venv\\' } |
  Select-String -Pattern '^\s*VITE_.*(KEY|SECRET|TOKEN|PASSWORD|PRIVATE)'
```

Any match needs review. A VITE_ variable is public by design; it must not hold a private credential.

8.3 Inspect production build output

```
# From the frontend project directory
npm run build

# Search built assets for suspicious names or known test markers
Get-ChildItem .\dist -Recurse -File |
  Select-String -Pattern
  'sk_live_|BEGIN PRIVATE KEY|DATABASE_URL|CLIENT_SECRET|VENDOR_API_KEY'
```

A clean string search does not prove the build is secret-free: secrets can be encoded, transformed, or have different names. Review source and build output, and use automated secret scanning.

8.4 Scan Git history

```
git status
git log --all --oneline
# Recommended: use a reputable secret scanner such as Gitleaks
gitleaks detect --source . --redact
```

A secret removed from the current files may remain in Git history. If a real secret was committed, revoke or rotate it immediately. History rewriting alone does not make the exposed credential safe.

9. What to Do if a Secret Was Exposed

• Revoke or rotate the exposed API key, password, token, or signing secret immediately.

• Update the backend environment or secret manager with the replacement value.

• Redeploy/restart affected services and verify the old credential no longer works.

• Inspect provider audit logs and application logs for suspicious use.

• Remove the secret from current source and, where appropriate, clean Git history in coordination with collaborators.

• Assume that deleting a secret from a public bundle or Git commit does not undo prior exposure.

Do not wait until the code cleanup is complete before rotating a credential that may have been exposed.

10. Additional Controls

Control	Recommendation
Least privilege	Create service-specific credentials with only the permissions required.
Secret manager	For production, prefer a managed secrets store or deployment-platform secret configuration.

Control	Recommendation
Network boundary	Keep PostgreSQL inaccessible from the public browser and restrict database network access.
HTTPS	Use HTTPS for frontend-to-backend and backend-to-provider communication.
Logging	Never log authorization headers, raw tokens, passwords, or complete secret-bearing connection strings.
Access control	Protect backend endpoints; moving a key to FastAPI does not mean every user should automatically be allowed to invoke the service.
Rotation	Define a process and schedule for rotating long-lived credentials.

11. Findings and Status for BugPilot

ID	Potential issue / review item	Status based on supplied evidence
S-01	Private API key in React source	Not verified – source tree not supplied for scan.
S-02	Private value in VITE_ variable	Not verified – frontend .env files not supplied.
S-03	Database credentials in client bundle	Not verified – build output not supplied.
S-04	Tokens in localStorage/sessionStorage	Not verified in this secrets-focused scan.
S-05	Backend secrets loaded from environment	Previously discussed, but current config was not supplied for validation.
S-06	.env excluded from Git	Not verified – current .gitignore and Git history not supplied.
S-07	Secret present in Git history	Not verified – repository history not supplied.

Important: These are scan targets and evidence gaps, not confirmed findings that BugPilot currently exposes secrets. No actual API key, private token, or password was identified in the code available for this report.

12. Verification Checklist

- ■ No private API keys, passwords, database URLs, or signing secrets exist in React source.
- ■ No VITE_ variable contains a private credential.
- ■ React calls BugPilot FastAPI for private third-party operations.
- ■ FastAPI reads private values from backend environment configuration or a secret manager.
- ■ Actual .env files are excluded from Git.
- ■ .env.example contains placeholders only.
- ■ Production build output has been inspected.
- ■ Git history has been scanned for previously committed secrets.
- ■ Any exposed credential has been revoked and rotated.
- ■ Backend endpoints using private credentials enforce authentication, authorization, input validation, and rate limits.

13. Conclusion

The central security principle is simple: anything shipped to the browser is public to the user. Vite's VITE_ prefix is an exposure mechanism for public frontend configuration, not a secure vault. Private API keys, database credentials, signing secrets, and private tokens must remain on the backend or in a managed secrets service.

For BugPilot, the recommended architecture is to keep React limited to public configuration such as the API base URL, route privileged operations through FastAPI, load private credentials server-side, protect environment files from Git, and scan both source code and build artifacts.

Assessment status: This is a preliminary security review and implementation guide. A definitive scan result requires the actual React source, environment configuration with secret values redacted, build output, `.gitignore`, and repository history.